Project Documentation: SecureDoc-RAG

1. Introduction

SecureDoc-RAG is an enterprise-grade document query system built with Retrieval-Augmented Generation (RAG). It enables multi-tenant document management where users can upload confidential documents and receive precise answers powered by Google Gemini 2.5 Flash and Pinecone vector database. The system is designed with strict data isolation, automated Personal Identifiable Information (PII) masking, and robust anti-prompt-injection safeguards.

2. Core Features & Capabilities

- **Multi-Tenant Architecture**: Supports multiple users with complete data isolation. Cross-tenant data leakage is prevented at the vector database level.
- **Automated PII Masking**: Sensitive information such as emails, passwords, phone numbers, credit cards, and IDs (PAN, Aadhaar) are automatically redacted before processing or embedding.
- **RAG-Powered AI Q&A**: Uses state-of-the-art LLMs to answer natural language questions based directly on the uploaded documents.
- **Anti-Prompt Injection Defense**: Untrusted document contents are securely wrapped in system boundaries to prevent malicious instructions from hijacking the AI model.

3. System Architecture

The system architecture follows a secure and isolated pipeline: 1. **Document Upload**: Users upload PDF or DOCX files. 2. **Text Extraction & Masking**: Text is extracted using PyMuPDF and python-docx. The text is immediately passed through a PII Masking Engine. 3. **Chunking & Embedding**: Cleaned text is chunked using LangChain and converted into vector embeddings via Gemini. 4. **Vector Storage**: Embeddings are stored in Pinecone with strict metadata tagging (`user_id` and `document_id`). 5. **Query & Retrieval**: User questions are embedded, and a vector search is performed with a mandatory `user_id` filter. 6. **Generation**: The retrieved context and question are sent to the Gemini model to generate a secure, accurate response.

4. Security & Data Isolation

The primary focus of SecureDoc-RAG is data security. - **Mandatory Pinecone Tenant Isolation**: Every chunk indexed stores strict metadata. Vector retrieval always applies a mandatory metadata filter (`{"user_id": {"$eq": "user_id"}}`), making cross-tenant search impossible. - **Regex PII Masking Engine**: Data is sanitized on the fly. Example transformations: - Emails ➔ `[EMAIL]` - Credit Cards ➔ `[CARD]` - Passwords ➔ `[PASSWORD]` - Aadhaar Cards ➔ `[AADHAAR]`

5. Technology Stack

Frontend

- **Framework**: React 19 + Vite
- **Styling**: Tailwind CSS v4
- **State Management**: TanStack Query (React Query) v5

Backend

- **Framework**: FastAPI (Python 3.12+)
- **Database**: SQLAlchemy 2.0 (SQLite / PostgreSQL)
- **Authentication**: JWT & Passlib bcrypt

AI & Data Pipeline

- **LLM / Embeddings**: Google Gemini 2.5 Flash / `text-embedding-004`
- **Vector DB**: Pinecone Serverless
- **Chunking**: LangChain `RecursiveCharacterTextSplitter`

6. Setup & Deployment

The system requires Node.js 18+ and Python 3.10+. - **Environment**: Keys required include `GEMINI_API_KEY`, `PINECONE_API_KEY`, and `JWT_SECRET`. - **Backend Start**: `uv run uvicorn backend.app.main:app --reload --port 8000` - **Frontend Start**: `cd frontend && npm install && npm run dev`

7. API Reference Summary

- `POST /api/auth/register` - Register a new tenant.
- `POST /api/auth/login` - Obtain JWT token.
- `POST /api/documents/upload` - Securely upload and process documents.
- `POST /api/chat` - Ask questions against the tenant's isolated RAG vector index.
- `GET /api/documents/{id}/view` - View PII-masked document content.

8. Testing & Validation

The project includes a robust Pytest suite testing PII masking logic, tenant vector isolation, and API endpoint security to ensure production-level reliability.